Project 1 Report: Semantic Segmentation

1. Introduction

This project implements a CNN-based semantic segmentation model for Pascal-VOC style 21-class pixel-level prediction. The goal of semantic segmentation is to classify every pixel in an input image into one of the semantic classes, including the background class. The project was designed under the given constraints: only CNN-based models were used, no pretrained semantic segmentation weights were loaded, and no validation or test annotations were used for training.

The final implementation uses PyTorch and TorchVision. The backbone is initialized from TorchVision EfficientNet-B3 ImageNet-1K classification pretrained weights, while the segmentation neck and head are implemented manually. The overall goal was to balance segmentation accuracy and computational efficiency, since the final score considers both mIoU and FLOPs.

2. Model Architecture

The final model is an FPN-like semantic segmentation network built on top of EfficientNet-B3. The model follows a Backbone-Neck-Head structure.

The backbone extracts multi-scale feature maps from EfficientNet-B3. Four feature levels are used: low-level, mid-level, and high-level feature maps corresponding approximately to strides 4, 8, 16, and 32. These multi-scale features are fused in a top-down manner, similar to Feature Pyramid Network architecture.

The final model can be summarized as follows:

Component	Design
Backbone	TorchVision EfficientNet-B3
Pretraining	ImageNet-1K classification weights
Neck	FPN-like multi-scale feature fusion
Feature levels	c2, c3, c4, c5
FPN channels	96
Head	Depthwise separable segmentation head
Output classes	21 classes: 20 Pascal-VOC classes + background
Output format	Class-index PNG prediction

This architecture satisfies the project requirement because it is fully CNN-based and does not use RNN, Transformer, or pretrained semantic segmentation models.

3. Dataset and Preprocessing

The model was trained using Pascal-VOC and additional MS-COCO data converted into Pascal-VOC style labels.

The training datasets are:

Dataset	Split	Usage
Pascal-VOC 2007	Train	Training
Pascal-VOC 2012	Train	Training
MS-COCO	train2017 subset converted to VOC classes	Additional training

The validation dataset is:

Dataset	Split	Usage
Pascal-VOC 2012	Val	Validation only

Validation data was not used for training. It was used only for validation mIoU computation and checkpoint selection. Pixels with ignore label 255 were excluded from both the cross-entropy loss and mIoU computation.

Input images were resized and cropped to 512 × 512. The standard ImageNet normalization statistics were used:

```
mean = [0.485, 0.456, 0.406]
std  = [0.229, 0.224, 0.225]
```

The implemented data augmentation pipeline includes horizontal flipping, random scaling, random cropping, color jitter, and copy-paste augmentation. The final configuration uses the following main augmentation settings:

Augmentation	**Value**
Horizontal flip probability	0.5
Random scale probability	1.0
Scale range	0.5 to 2.0
Random crop probability	1.0
Copy-paste probability	0.2
Color jitter probability	0.4
Stitching probability	0.0

Stitching augmentation was disabled in the final configuration. Because of RAM issue.

4. Training Recipe

The model was trained on Google Colab using CUDA GPU acceleration. The project was mainly designed for Colab T4, L4, A100 environments. In most cases L4 has been used, however while training EfficientNet-B3 with AdamW, I used A100 for almost 16 hours.

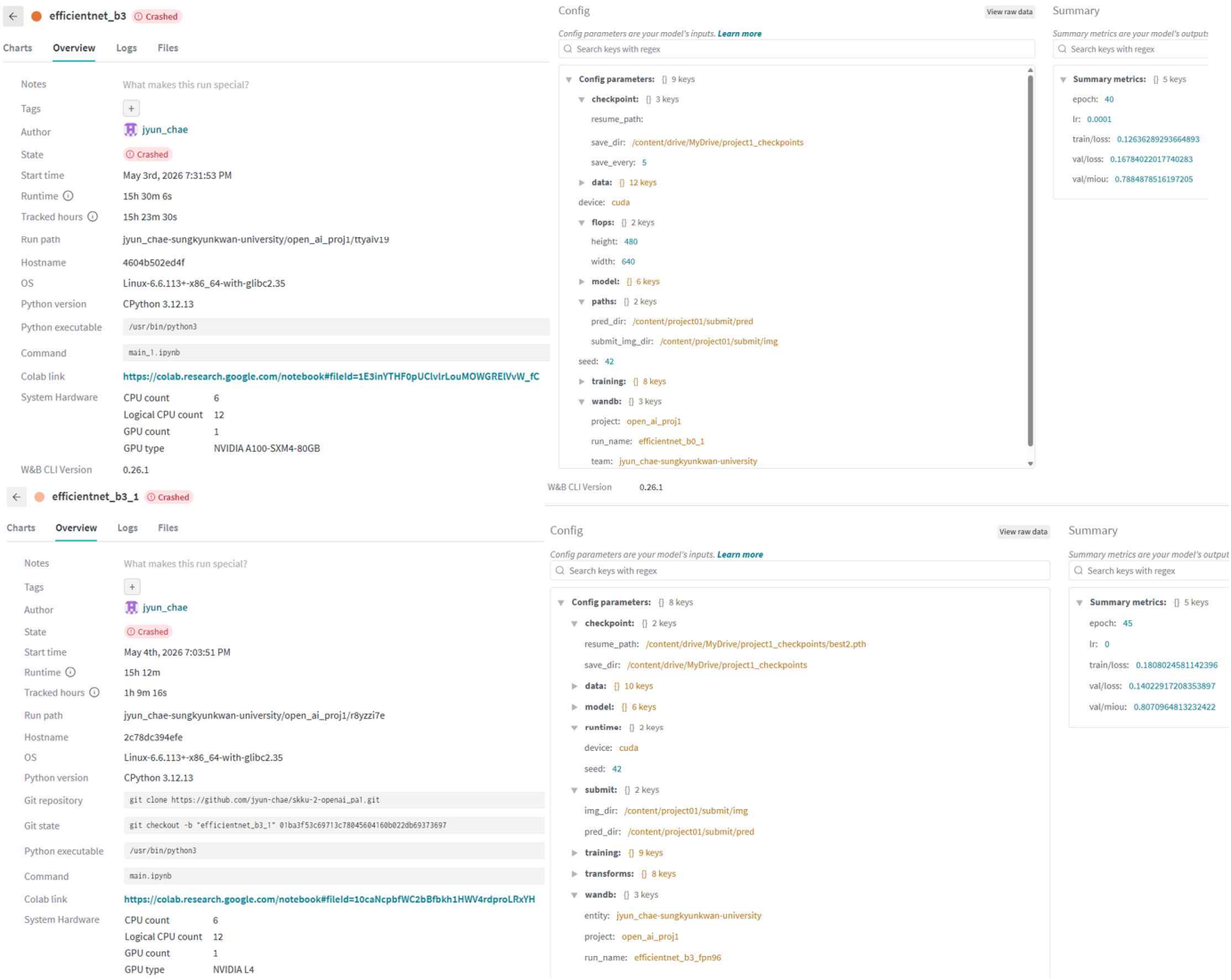
The final training configuration was:

Hyperparameter	**Value**
Input size	512
Batch size	16
Epochs	30
Optimizer	SGD-style parameter groups
Backbone learning rate	1.0e-4
Head / neck learning rate	1.0e-3
Weight decay	1.0e-4
Scheduler	Exponential decay
LR decay	0.9995
AMP	Enabled
Evaluation interval	Every 2 epochs
Seed	42

Different learning rates were used for the pretrained backbone and newly initialized segmentation layers. The backbone learning rate was set lower to avoid destroying pretrained ImageNet features too quickly, while the neck and head were trained with a larger learning rate.

Automatic Mixed Precision was enabled to reduce GPU memory usage and improve training speed. The training loop used gradient scaling when AMP was active.

Checkpoints were saved during training. The latest checkpoint was saved as last.pth, and the best validation checkpoint was saved as best.pth based on validation mIoU. This made training more robust against Colab runtime interruptions.



5. Evaluation Metric

The main evaluation metric is mean Intersection-over-Union, or mIoU. IoU is computed for each class and then averaged across valid classes.

The implementation accumulates a confusion matrix over the validation dataset. For each class, true positives, false positives, and false negatives are computed from the confusion matrix. Pixels with ignore index 255 are removed before updating the confusion matrix.

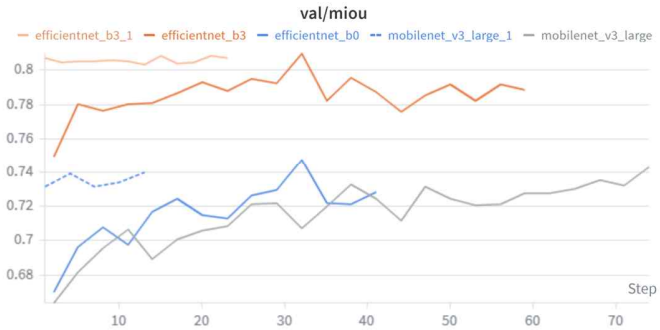
The metric implementation follows:

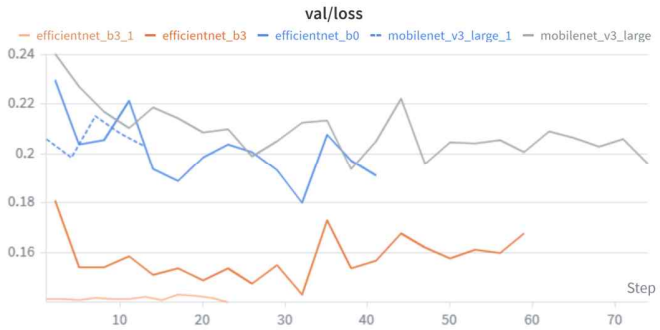
```
IoU = TP / (TP + FP + FN)
mIoU = mean IoU across valid classes
```

6. Validation Results

Model	Validation mIoU	Validation Loss	Notes

EfficientNet-B3 + AdamW	0.80982	0.14323	Low Epoch
EfficientNet-B3 + SGD	0.8071	0.14023	High Epoch (Final Model)





The best checkpoint was selected based on the validation mIoU during training. According to the training curves, the validation mIoU reached its peak at the selected checkpoint, while the validation loss began to increase after that point, indicating possible overfitting.

To further improve performance, I changed the optimizer to SGD and resumed training from best.pth. During this additional training phase, another checkpoint was saved at an appropriate point. Although the original best.pth achieved the best validation-set performance, the final test-set evaluation showed that the last checkpoint, trained for the largest number of epochs, achieved the highest mIoU.

7. Ablation and Trial History

Several model architectures and training strategies were tested before deciding on the final configuration. The experiments were mainly focused on finding a practical balance between segmentation accuracy, FLOPs, memory usage, and training stability.

7.1 Model Architecture Trials

The model architecture was gradually revised through multiple trials.

Trial	Change	Observation
ResNet-based FPN	Used ResNet as the backbone	The representation capacity was strong, but the FLOPs were too high for the project constraint.
MobileNet-based FPN	Replaced ResNet with a lightweight MobileNet backbone	The FLOPs became sufficiently small, but the model was too lightweight and showed limited segmentation performance.
EfficientNet-B0 backbone	Tested EfficientNet-B0 as a more balanced backbone	It provided better features than MobileNet, but the performance was still not sufficient.
Neck modification for FLOPs reduction	Adjusted the FPN neck structure to reduce computation	FLOPs decreased, but excessive reduction in the neck capacity caused performance degradation.
EfficientNet-B3 backbone	Switched to a stronger EfficientNet-B3 backbone	The feature quality improved, but the FLOPs became too high.
Reduced feature extraction depth	Reduced the number or depth of extracted backbone features	This lowered FLOPs, but also reduced the quality of high-level semantic features and hurt performance.

Depthwise convolution in high-level neck	Applied depthwise separable convolutions to the high-level FPN neck	This reduced FLOPs while preserving more of the representational power of EfficientNet-B3.

Based on these trials, the final model uses EfficientNet-B3 as the backbone. ResNet-based models were too expensive, while MobileNet and EfficientNet-B0 were not strong enough for the target segmentation performance. EfficientNet-B3 provided better feature representations, but required additional optimization to satisfy the FLOPs constraint. Therefore, depthwise separable convolutions were introduced in the high-level neck to reduce computational cost while maintaining the benefit of a stronger backbone.

7.2 Data Augmentation Trials

The data augmentation strategy was also revised step by step.

Trial	Change	Observation
Basic transforms	Used simple preprocessing and normalization	Stable, but the diversity of training samples was limited.
Standard augmentations	Added horizontal flip, random scaling, cropping, and color-related augmentations	Improved data diversity and helped the model generalize better.
Random source sampling	Added random image and mask selection functions for copy-paste and stitching	Enabled more advanced object-level and region-level augmentation.
Stitching augmentation	Combined regions from different images	Caused instability and excessive RAM usage during training.
Final augmentation setting	Disabled stitching by setting its probability to 0	Improved training stability while keeping safer augmentations active.

The final data augmentation pipeline uses standard augmentations such as horizontal flip and random scale, together with carefully controlled copy-paste augmentation. Although stitching was implemented and tested, it was disabled in the final configuration because it caused RAM-related instability during training. This final setting provided a more stable balance between data diversity and runtime reliability.

8. Inference and Test-Time Augmentation

The output stores class indices in the range [0, 20].

The final configuration supports test-time augmentation. Horizontal flip TTA and multi-scale inference are enabled. The configured inference scales are:

```
[0.50, 0.625, 0.75, 0.875, 1.00, 1.125, 1.25, 1.50, 1.75, 2.00]
```

Predictions from multiple scales are averaged using probability averaging. This improves robustness, although it increases inference time.

9. ONNX

The project includes ONNX export code for FLOPs measurement. The model is exported using a dummy input and saved as:

```
submit/model.onnx
```

10. Use of AI Tools

AI tools were used as supportive resources throughout the project. I used ChatGPT to discuss model selection and to refine the design of the neck structure. ChatGPT was also used to troubleshoot practical issues related to WandB integration, GitHub repository management, and Colab-specific environment problems.

In addition, I discussed suitable data augmentation strategies for image segmentation with ChatGPT and used it to help implement and debug augmentation methods. For code cleanup, minor refactoring, and small syntax or implementation issues, I also used GitHub Copilot in VS Code.

However, the final implementation decisions, training execution, validation checks, WandB result interpretation, and final model selection were manually reviewed and verified. AI tools were used only as assistants, while the key experimental judgments and conclusions were made independently.

11. Conclusion

This project implemented a CNN-based semantic segmentation system for Pascal-VOC style 21-class prediction. The final model uses a TorchVision EfficientNet-B3 ImageNet-1K pretrained classification backbone, an FPN-like feature fusion neck, and a lightweight depthwise separable segmentation head.

The implementation follows the project constraints: it does not use pretrained semantic segmentation weights, does not use Transformer or RNN layers, and does not use validation or test annotations for training. The training pipeline includes checkpoint saving, validation mIoU computation, WandB logging, inference output generation, and ONNX export for FLOPs measurement.

The final design focuses on balancing segmentation performance and computational efficiency. EfficientNet-B3 provides strong CNN features, while depthwise separable FPN blocks reduce the computational cost of multi-scale feature fusion.

Original Overview (Non cut)